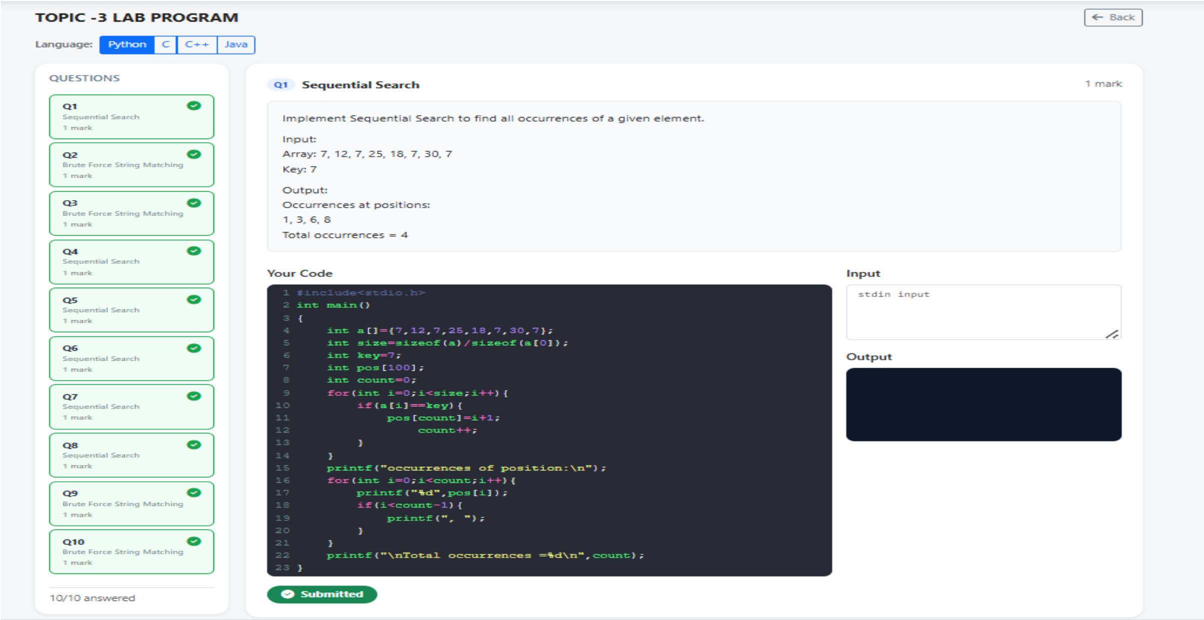
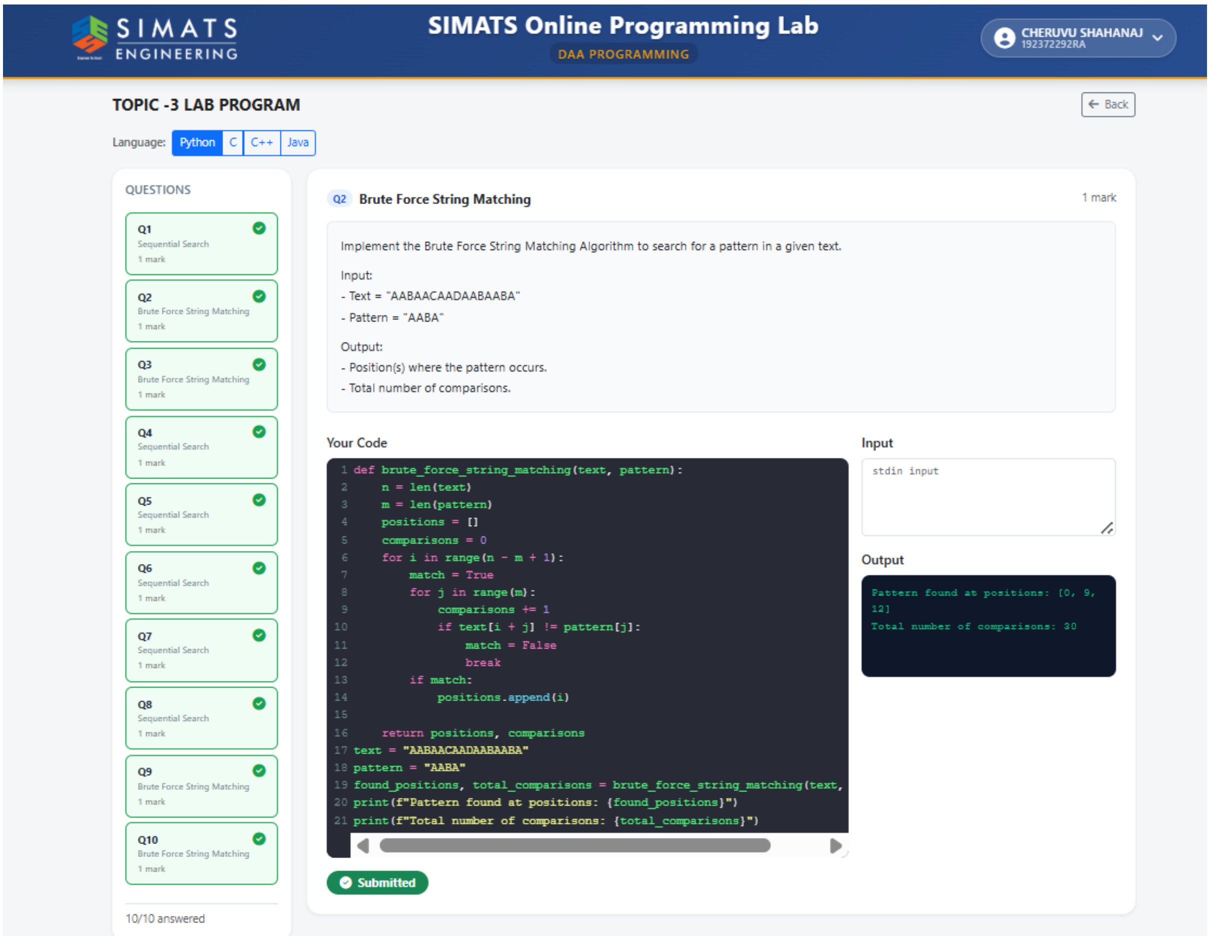
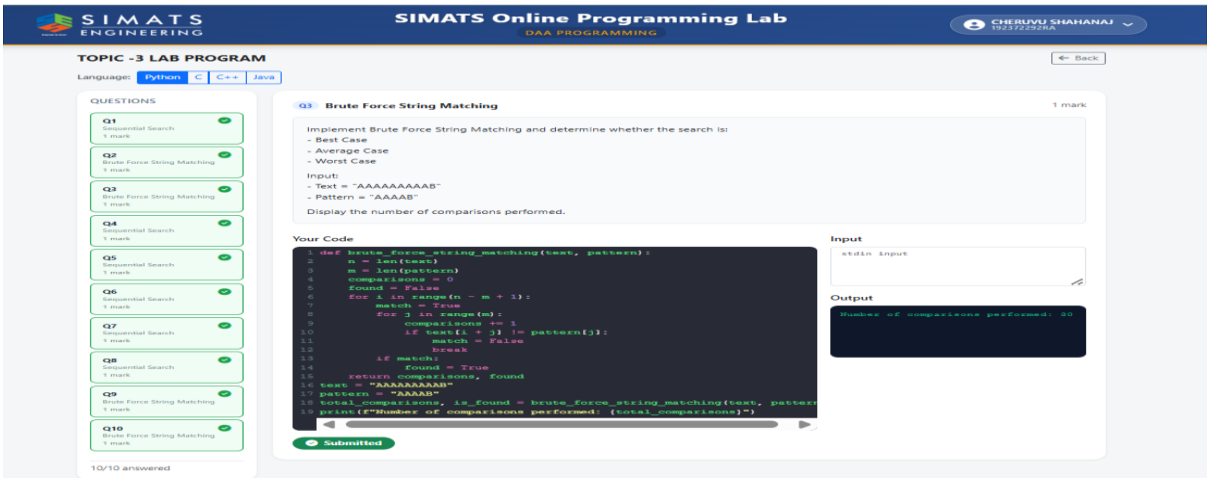
LAB PROGRAM -03

SHAHANAJ C

192372292







SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ 192372292RA

TOPIC -3 LAB PROGRAM

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 Sequential Search 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Sequential Search 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Sequential Search 1 mark
Q9 Brute Force String Matching 1 mark
Q10 Brute Force String Matching 1 mark

10/10 answered

Q3 Brute Force String Matching

1 mark

Implement Brute Force String Matching and determine whether the search is:
- Best Case
- Average Case
- Worst Case

Input:
- Text = "AAAAAAAAAB"
- Pattern = "AAAAB"

Display the number of comparisons performed.

Your Code

```
def brute_force_string_matching(text, pattern):
    n = len(text)
    m = len(pattern)
    comparisons = 0
    found = False
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        if match:
            found = True
    return comparisons, found
text = "AAAAAAAAAB"
pattern = "AAAAB"
total_comparisons, is_found = brute_force_string_matching(text, patter
print(f"Number of comparisons performed: {total_comparisons}")
```

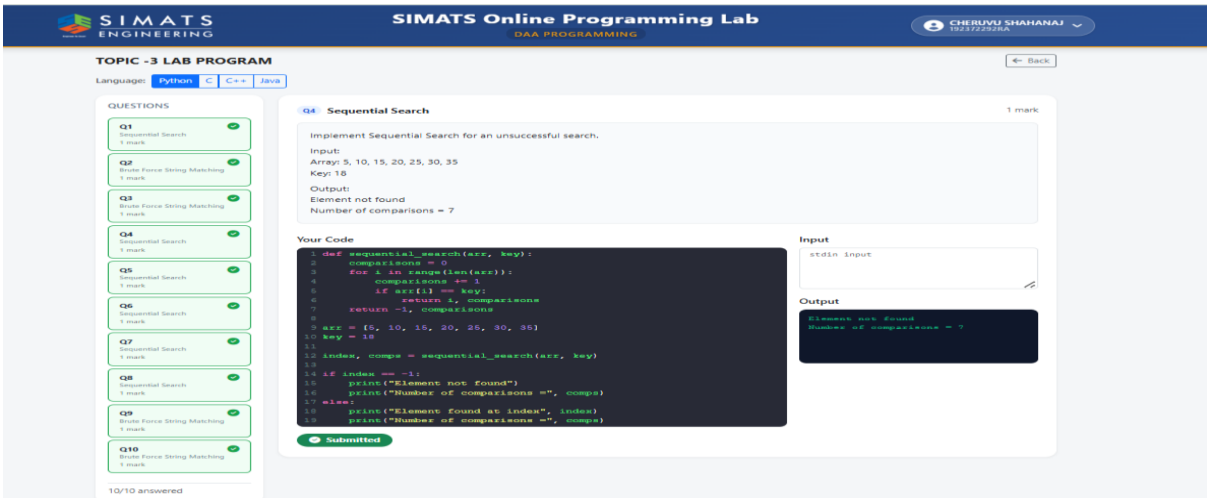
Input

stdin input

Output

```
Number of comparisons performed: 30
```

Submitted



SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ 192372292RA

TOPIC -3 LAB PROGRAM

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 Sequential Search 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Sequential Search 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Sequential Search 1 mark
Q9 Brute Force String Matching 1 mark
Q10 Brute Force String Matching 1 mark

10/10 answered

Q4 Sequential Search

1 mark

Implement Sequential Search for an unsuccessful search.

Input:
Array: 5, 10, 15, 20, 25, 30, 35
Key: 18

Output:
Element not found
Number of comparisons = 7

Your Code

```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i, comparisons
    return -1, comparisons

arr = [5, 10, 15, 20, 25, 30, 35]
key = 18

index, comps = sequential_search(arr, key)

if index == -1:
    print("Element not found")
    print("Number of comparisons =", comps)
else:
    print("Element found at index", index)
    print("Number of comparisons =", comps)
```

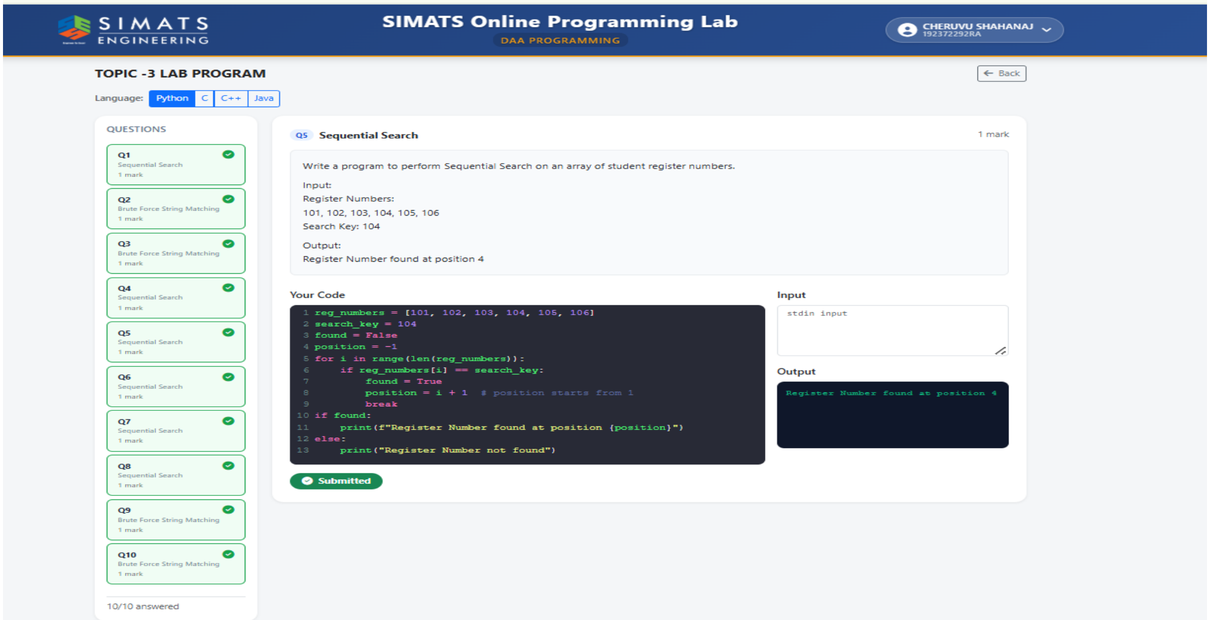
Input

stdin input

Output

```
Element not found
Number of comparisons = 7
```

Submitted



SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ 192372292RA

TOPIC -3 LAB PROGRAM

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 Sequential Search 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Sequential Search 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Sequential Search 1 mark
Q9 Brute Force String Matching 1 mark
Q10 Brute Force String Matching 1 mark

10/10 answered

Q5 Sequential Search

1 mark

Write a program to perform Sequential Search on an array of student register numbers.

Input:
Register Numbers:
101, 102, 103, 104, 105, 106
Search Key: 104

Output:
Register Number found at position 4

Your Code

```
reg_numbers = [101, 102, 103, 104, 105, 106]
search_key = 104
found = False
position = -1
for i in range(len(reg_numbers)):
    if reg_numbers[i] == search_key:
        found = True
        position = i + 1  # position starts from 1
        break
if found:
    print(f"Register Number found at position {position}")
else:
    print("Register Number not found")
```

Input

stdin input

Output

```
Register Number found at position 4
```

Submitted

SIMATS ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

TOPIC -3 LAB PROGRAM

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Sequential Search 1 mark
- Q5 Sequential Search 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Sequential Search 1 mark
- Q9 Brute Force String Matching 1 mark
- Q10 Brute Force String Matching 1 mark

10/10 answered

Q6 **Sequential Search** 1 mark

Implement Sequential Search and analyze its performance.

Input:
Array:
45, 23, 67, 12, 89, 34, 56, 78, 90, 11,
29, 73, 18, 64, 37

Search Keys:
73
18
100

Tasks:
- Show each comparison.
- Count comparisons for every search.
- Identify best, average, and worst cases.
- Determine:
- Best-case complexity
- Average-case complexity
- Worst-case complexity
- Space complexity

Your Code

```python
def sequential_search(arr, key):
    comparisons = 0
    found = False
    print(f"Searching for key: {key}")
    for i in range(len(arr)):
        comparisons += 1
        print(f"Comparison {comparisons}: Is {arr[i]} == {key}? ")
        if arr[i] == key:
            print(f"-> Match found at index {i}!")
            found = True
            break
        else:
            print(f"-> No match.")

    if not found:
```

Input

stdin input

Output

```
--- Execution Trace ---
Searching for key: 73
Comparison 1: Is 45 == 73?
-> No match.
Comparison 2: Is 23 == 73?
```

SIMATS ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

TOPIC -3 LAB PROGRAM

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Sequential Search 1 mark
- Q5 Sequential Search 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Sequential Search 1 mark
- Q9 Brute Force String Matching 1 mark
- Q10 Brute Force String Matching 1 mark

10/10 answered

Q7 **Sequential Search** 1 mark

Implement Sequential Search for string data.

Input:
Names:
Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:
Deepa

Output:
Name found at position 4

Your Code

```python
def sequential_search(names, target):
    for i in range(len(names)):
        if names[i] == target:
            return i + 1  # position (1-based index)
    return -1

# Input
names = input().split(", ")
search_name = input()

# Search
position = sequential_search(names, search_name)

# Output
if position != -1:
    print(f"Name found at position {position}")
else:
    print("Name not found")
```

Submitted

Input

Anu, Bala, Charan, Deepa,Esha, Farhan

Output

```
Traceback (most recent call last):
  File "C:\Windows\TEMP\sandbox_192372292RA_6a556edf4a4c30.12880210\main.py", line 9, in <module>
```

SIMATS ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

TOPIC -3 LAB PROGRAM

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Sequential Search 1 mark
- Q5 Sequential Search 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Sequential Search 1 mark
- Q9 Brute Force String Matching 1 mark
- Q10 Brute Force String Matching 1 mark

10/10 answered

Q8 **Sequential Search** 1 mark

Write a program to implement Sequential Search to find a given element in an array.

Input:
Array: 12, 25, 8, 45, 32, 19, 50
Key: 32

Output:
Element found at position 5
Number of comparisons = 5

Your Code

```python
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

arr = [12, 25, 8, 45, 32, 19, 50]
key = 32

position, comparisons = sequential_search(arr, key)

if position != -1:
    print(f"Element found at position {position}")
    print(f"Number of comparisons = {comparisons}")
else:
    print("Element not found")
    print(f"Number of comparisons = {comparisons}")
```

Submitted

Input

stdin input

Output

```
Element found at position 5
Number of comparisons = 5
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Sequential Search 1 mark
- Q5 Sequential Search 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Sequential Search 1 mark
- Q9 Brute Force String Matching 1 mark
- Q10 Brute Force String Matching 1 mark

10/10 answered

Q9 Brute Force String Matching

1 mark

Implement Brute Force String Matching for case-insensitive searching.

Input:
- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHMS"

Output:
- Pattern found position

Your Code

```python
def brute_force_match(text, pattern):
    text = text.lower()
    pattern = pattern.lower()
    n = len(text)
    m = len(pattern)

    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            return i
    return -1

text = "DataStructuresAndAlgorithms"
pattern = "ALGORITHMS"

position = brute_force_match(text, pattern)

if position != -1:
    print("Pattern found at position:", position)
else:
    print("Pattern not found")
```

Input

stdin input

Output

```
Pattern found at position: 17
```

Submitted

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

TOPIC -3 LAB PROGRAM

Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Sequential Search 1 mark
- Q5 Sequential Search 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Sequential Search 1 mark
- Q9 Brute Force String Matching 1 mark
- Q10 Brute Force String Matching 1 mark

10/10 answered

Q10 Brute Force String Matching

1 mark

Implement Brute Force String Matching and display every alignment of the pattern with the text.

Input:
- Text = "ABCDABCABCDA"
- Pattern = "ABCDA"

Output:
- Alignment number
- Matching result
- Pattern occurrence positions

Your Code

```python
text = "ABCDABCABCDA"
pattern = "ABCDA"

n = len(text)
m = len(pattern)

print("Text:", text)
print("Pattern:", pattern)
print()

found_positions = []

for i in range(n - m + 1):
    window = text[i:i+m]
    match = (window == pattern)

    print(f"Alignment {i} (Text index {i}-{i+m-1})")
    print(f"Pattern: {pattern}")
    print(f"Text    : {window}")
    print(f"Match   : {'YES' if match else 'NO'}")

    if match:
        found_positions.append(i)
        print(f"Occurrence positions: {i}")

    print()
```

Input

stdin input

Output

```
Match   : YES
Occurrence positions: 7

Final occurrence positions: [0,
7]
```